

材料データベースのための Schema Graph 支援 Text-to-SQL 手法：

正規化リレーショナル材料データへの自然言語クエリ手法

— OQMD 金属間化合物データ (B2 および L1₂ 型) による検証 —

皆本 悟

国立研究開発法人 物質・材料研究機構 (NIMS)

〒 305-0047 茨城県つくば市千現 1-2-1

Abstract

Open Quantum Materials Database (OQMD)、Materials Project、AFLOW などの材料データベースには、第一原理計算による膨大な物性データが蓄積されているが、正規化されたリレーショナルスキーマを通じたデータアクセスには多くの材料研究者が持ち合わせていない SQL (Structured Query Language) の知識が必要である。本研究では、正規化された材料データベースに対し、自然言語クエリを実行可能な SQL 文へ自動変換する **Schema Graph** 支援 **Text-to-SQL** (**T2SQL**) 手法を提案する。

本手法は 4 つの技術要素から構成される：(1) 日英バイリンガル材料用語辞書を備えたドメイン特化型条件抽出器、(2) 外部キー (FK) 関係を NetworkX グラフとして表現し、Steiner 木近似により最小 JOIN 経路を計算する **Schema Graph** 走査エンジン、(3) 成功した NL-SQL ペアを蓄積し、TF-IDF コサイン類似度で検索して LLM プロンプトに注入する **Few-Shot** 検索拡張生成 (**RAG**) ストア (SQL-as-Few-Shot-Examples, LaTeX 原稿からの種事例自動抽出を含む)、および (4) キーワードブラックリスト、単一文制約、テーブルホワイトリスト、自動 LIMIT 注入を行う多層 **SQL** ガード。

909 件の OQMD 金属間化合物 (B2 型 CsCl 636 件、L1₂ 型 AuCu₃ 273 件) に対し、正常クエリ、該当なしクエリ、曖昧・不正入力、矛盾条件、SQL インジェクション攻撃、安全性検査を含む 39 件のテストケースで検証を行った。3 水準比較 (Schema Graph なし Naive T2SQL / Schema Graph + Rule-based / Schema Graph + GPT-5) により、Schema Graph 走査が不要な JOIN を排除し、多元素 AND 検索 (Naive アプローチでは 0 件となる) を正しく処理し、OQMD API 直接取得による正解データに対して Precision = 1.0 を達成することを示した。Rule-based モードは外部 API 依存ゼロで 100 ms 未満のレイテンシを実現し、GPT-5 モードは約 7.3 秒のレイテンシとトークン消費の代償で、辞書未登録語彙や数値フィルタ生成への対応力を提供する。

AI for Science (AI4S) のパラダイムにおいて、AI が仮説生成からデータ検索、実験設計に至る科学ワークフローのあらゆる段階を加速すると期待される中、構造化された材料データへの自然言語アクセスは重要な基盤技術である。本手法は材料インフォマティクスにおけるデータ活用の民主化に貢献し、AI4S 駆動型材料探索の基礎的構成要素を提供する。

キーワード：Text-to-SQL、材料データベース、スキーマグラフ、検索拡張生成 (RAG)、OQMD、金属間化合物、自然言語処理、マテリアルズ・インフォマティクス、AI for Science

Contents

1	緒言	3
1.1	AI for Science と材料インフォマティクスにおけるデータアクセス障壁	3
1.2	Text-to-SQL の技術動向	3
1.3	材料インフォマティクスにおける NLP	3
1.4	目的と新規性	3
2	手法	4
2.1	材料データのためのデータベーススキーマ設計	4
2.1.1	OQMD データの 7 テーブルスキーマへの正規化	4
2.1.2	RAG コンテキスト注入のための XML/YAML 表現	5
2.2	材料用語辞書を備えた条件抽出器	5
2.2.1	辞書構造	5
2.2.2	Unicode 正規化とパターンマッチング	6

2.2.3	出力形式	6
2.3	Schema Graph 走査エンジン	6
2.3.1	PostgreSQL メタデータからのグラフ構築	6
2.3.2	最小被覆 JOIN 経路: Steiner 木近似	6
2.3.3	JOIN 経路 → SQL FROM/JOIN 句生成	7
2.3.4	多元素 AND クエリ: EXISTS 副問い合わせパターン	7
2.3.5	比較: Naive vs. Schema Graph (具体例)	7
2.4	SQL 生成: Rule-based モードと LLM モード	8
2.4.1	Rule-based モード (Level 1)	8
2.4.2	Few-Shot RAG を備えた LLM モード (Level 2)	8
2.5	Few-Shot RAG ストア (SQL-as-Few-Shot-Examples)	8
2.5.1	アーキテクチャ	8
2.5.2	材料クエリのための TF-IDF トークナイゼーション	8
2.5.3	研究論文からの種事例抽出	9
2.6	SQL ガード: 多層安全性検証	9
2.7	推奨運用構成: カスケードモード	9
3	データ準備	9
3.1	OQMD API からのデータ取得	9
3.2	RDB へのデータ投入	10
3.3	スキーマ拡張	10
4	結果	10
4.1	テストケース設計	10
4.2	3 水準比較	10
4.3	Rule-based vs. GPT-5 詳細比較	11
4.3.1	結果セッター一致度 (Jaccard 指数)	11
4.3.2	注目すべき乖離	11
4.3.3	レイテンシとコスト	11
4.4	OQMD API 正解データによる検証 (結合テスト)	11
4.5	SQL インジェクションと安全性結果	13
5	考察	13
5.1	多次元評価	13
5.2	各水準の利点と欠点	15
5.3	限界と制約	15
5.3.1	テストケースの自己参照性 (深刻度: 高)	15
5.3.2	Few-Shot RAG 効果の未検証 (深刻度: 高)	15
5.3.3	無通知失敗 (深刻度: 中)	15
5.3.4	小規模スキーマ (深刻度: 中)	15
5.3.5	Rule-based モードの数値条件制限 (深刻度: 中)	15
5.4	関連システムとの比較	15
5.5	AI for Science (AI4S) における意義	15
5.6	今後の展望	16
6	結論	16
A	全テストケース一覧	19
B	生成 SQL 例	20
B.1	A01: 「Fe を含む B2 化合物を出して」	20
B.2	A03: 「Ni と Al の両方を含む化合物を出して」	20

1 緒言

1.1 AI for Science と材料インフォマティクスにおけるデータアクセス障壁

AI for Science (AI4S) —科学的発見を加速するための人工知能の体系的応用—は、材料研究を変革しつつある [Wang et al., 2023]。AI4S は、AI エージェントが自律的に仮説を生成し、大規模データベースから関連データを検索し、実験やシミュレーションを設計し、結果を解釈する統合的ワークフローを構想している。このビジョンにおいて、構造化された材料データへの自然言語アクセスは単なる利便性ではなく前提条件である：AI エージェント（またはそれを指揮する研究者）が正規化リレーショナルデータベースを効率的にクエリできなければ、AI4S パイプライン全体がデータ検索段階でボトルネックとなる。

ハイスループット第一原理計算の急速な発展により、複数の大規模材料データベースが構築されている：Open Quantum Materials Database (OQMD, 約 100 万件) [Saal et al., 2013, Kirklin et al., 2015]、Materials Project (15 万件超) [Jain et al., 2013]、AFLOW (350 万件超) [Curtarolo et al., 2012]、NOMAD [Draxl and Scheffler, 2018]。これらのデータベースは計算材料探索、候補相のスクリーニング [Senkov et al., 2018, George et al., 2019]、実験観測の検証に不可欠である。

しかし、正規化リレーショナルスキーマ（外部キーで接続された複数テーブル）に格納されたデータへのアクセスには SQL 知識が必要である。例えば、「Fe を含む安定な B2 化合物の格子定数」を適切に正規化されたスキーマで検索するには、元素・プロトタイプ・熱力学的安定性に対する WHERE 条件を持つ 4 テーブル JOIN が必要となる—データベース技術者にとっては容易だが、材料研究者にとっては非自明な操作である。

REST API (OQMD の /oqmdapi/formationenergy など) はこの障壁を部分的に緩和するが、独自の学習コスト（フィルタ構文、ページネーション処理、レスポンス解析）を課し、ローカルに構築したデータセットに対する柔軟な SQL クエリの能力を欠いている。

AI4S の文脈において、このデータアクセス障壁は特に深刻である：材料スクリーニングや物性予測を行う自律 AI エージェントは、複雑な複合条件データベースクエリをプログラマ的に発行する必要がある。正しい SQL を確実に生成する自然言語インターフェースは、人間の研究者と AI エージェントの双方がデータベース工学の専門知識なしに材料データベースと対話することを可能にし、AI4S ワークフローの重要なボトルネックを除去する。

1.2 Text-to-SQL の技術動向

Text-to-SQL (T2SQL) 研究は大規模ベンチマークにより急速に進展している：Spider [Yu et al., 2018] (10,181 問、200 データベース、138 ドメイン)、BIRD [Li et al., 2024] (12,751 問、95 データベース、33.4 GB)。最近の LLM ベース手法は高い精度を達成している：DAIL-SQL [Gao et al., 2023] はスケルトンベースの few-shot 事例選択により Spider 上で実行精度 86.6% を報告し、DIN-SQL [Pourreza and Rafiei, 2024] は分解型文脈内学習により 85.3% を達成している。Hong et al. [2024] はスキーマリンクの支配的役割を指摘する包括的サーベイを提供し、Maamari et al. [2024] は高度な LLM により明示的スキーマリンクが不要になる可能性を論じているが、これは議論が続いている。

しかし、既存の **T2SQL** ベンチマークはすべて汎用ドメイン（e コマース、医療、教育）を対象としており、材料科学データベース固有の課題に取り組んだ先行研究は存在しない：ドメイン固有の用語（Strukturbericht 記号、プロトタイプ名）、バイリンガルクエリ（日英元素名）、組成テーブル正規化（化合物ごとに元素 1 行）、熱力学的安定性フィルタ ($E_{\text{hull}} \leq 0.001$ eV/atom) などの課題である。

1.3 材料インフォマティクスにおける NLP

材料科学における NLP 応用は拡大している：MatSci-NLP [Song et al., 2023] は材料テキストの 7 つの NLP タスクをベンチマークし、LLAMAT [Mishra et al., 2024] は LLaMA の継続事前学習により情報抽出を改善し、Materials Knowledge Navigation Agent (MKNA) [Zhuang and Farimani, 2026] は自然言語の意図をデータベース検索・構造生成アクションに変換し、OptiMat Alloys [Hu and Turlo, 2026] は LLM 駆動型対話式合金探索ツールを提供している。Materials Project は MCP ベースの LLM 統合の提供を開始している。

これらの研究は情報抽出やエージェントベースのデータベース対話を扱っているが、正規化リレーショナルデータベース上のスキーマ認識型 **SQL** 生成—特に外部キーグラフ走査による自動 JOIN 経路探索—を対象としたものはない。このギャップが本手法の焦点である。

1.4 目的と新規性

本研究の貢献は以下の 3 点である：

1. 材料ドメイン特化型 **Schema Graph T2SQL** 手法：日英バイリンガル材料用語辞書と FK 関係グラフ走査を組み合わせ、正規化材料データベース上の自動 JOIN 経路計算を実現する。
2. **SQL-as-Few-Shot-Examples** と原稿ベース種事例抽出：成功した NL-SQL ペアを蓄積し TF-IDF 類似度で LLM プロンプトに注入する RAG フィードバックループ。LaTeX 研究論文からの自動種事例抽出を含む。
3. **OQMD API** 正解データを用いた結合テスト手法：データパイプライン全体（API → CSV → RDB 正規化 → NL → SQL → JOIN 再構成）の正確性を Precision/Recall で定量評価する。

2 手法

本節では各技術要素を詳述する。システム全体のパイプラインアーキテクチャを図 1 に示す。

Fig. 1: System Architecture — Schema-Graph-Assisted Text-to-SQL Pipeline

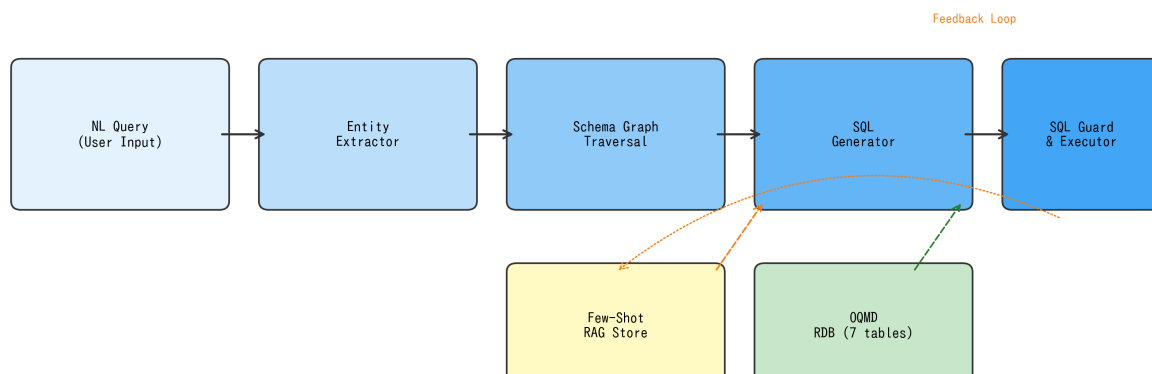


Figure 1: Schema Graph 支援 Text-to-SQL パイプラインのシステムアーキテクチャ。実線矢印は主要データフロー、破線矢印は Few-Shot RAG フィードバックループ（成功したクエリペアが蓄積・検索される）を示す。SQL ガードはデータベース実行前の最終安全層として機能する。

2.1 材料データのためのデータベーススキーマ設計

2.1.1 OQMD データの 7 テーブルスキーマへの正規化

OQMD の REST API はフラットな JSON レコード（1 エントリ = 1 JSON オブジェクト）を返す。これを 7 テーブルのリレーショナルスキーマに正規化する（図 2、表 1）。設計は Boyce-Codd 正規形（BCNF）に従い、以下の材料ドメイン固有の考慮に基づく。

Table 1: データベーススキーマ概要（7 テーブル）。

テーブル名	主キー	外部キー	主要カラム
material_entry	entry_id	—	formula, reduced_formula, chemical_system
composition	composition_id	entry_id	element, atomic_fraction
structure	structure_id	entry_id	prototype, strukturbericht, lattice_a, space_group
phase_stability	stability_id	entry_id	formation_energy_per_atom, energy_above_hull, band_gap
calculation	calculation_id	entry_id	method, functional
calculated_property	property_id	calculation_id	property_name, value, unit
prototype_definition	prototype_id	—	prototype_name, strukturbericht, description

組成テーブルの分離。単一の化合物が複数の元素を含む場合がある（例：Fe₃Al → Fe と Al）。各元素を composition テーブルの個別行として格納することで、文字列解析なしに柔軟な元素ベースクエリ（AND, OR, NOT）を可能にする。ただし、この正規化は重要な複雑性を導入する：「Ni と Al の両方を含む化合物

Fig. 2: Entity-Relationship Diagram — Materials Database Schema (7 Tables)

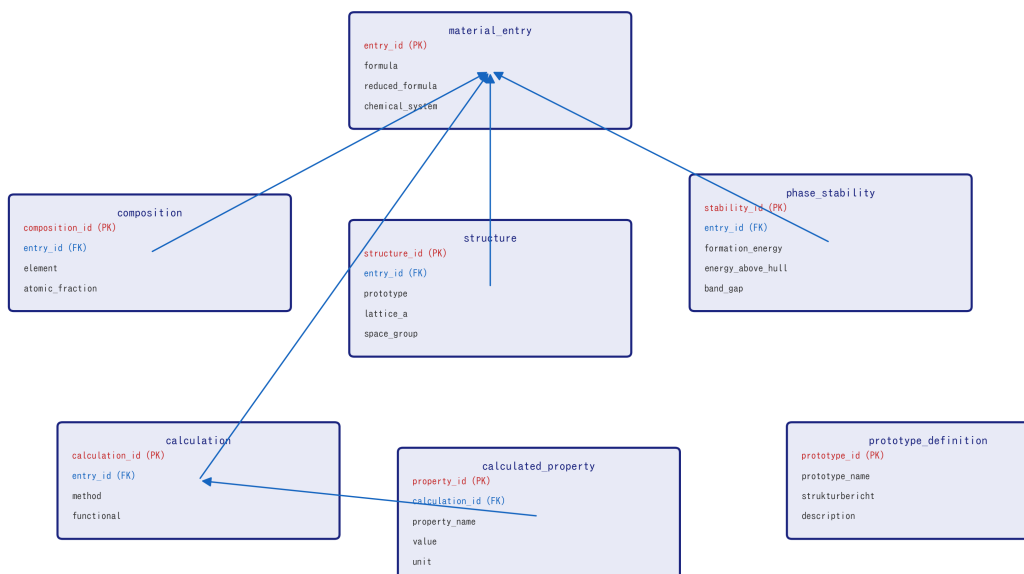


Figure 2: 7 テーブル材料データベーススキーマの Entity-Relationship 図。すべてのテーブルは外部キー（青線）により **material_entry** に接続される。赤=主キー、青=外部キー。

物」のクエリは単純な `WHERE element = 'Ni' AND element = 'Al'` では不可能である（単一行が両条件を同時に満たすことはないため 0 行が返る）。代わりに `EXISTS` 副問い合わせが必要となる（2.3.4 節参照）。

構造/相安定性/計算の分離。結晶構造情報（プロトタイプ、格子定数）、熱力学的安定性（hull 上エネルギー）、計算メタデータ（DFT 汎関数）を別テーブルに格納し、独立した更新・拡張を可能にする。

計算物性の **Entity-Attribute-Value (EAV)** パターン。 `calculated_property` テーブルは EAV パターン（`property_name`, `value`, `unit`）を使用し、スキーマ変更なしに任意の計算物性（体積弾性率、せん断弾性率等）を収容する。

2.1.2 RAG コンテキスト注入のための XML/YAML 表現

スキーマ構造は YAML (`allowed_schema.yaml`) としてシリアル化され、二重の目的に供される：(a) SQL ガードのテーブル/カラムホワイトリスト（2.6 節）、(b) LLM プロンプトへの RAG コンテキストスキーマ YAML がシステムメッセージに注入され、LLM が利用可能なテーブル・カラム・型を理解する。このアプローチは汎用 T2SQL のスキーマシリアル化ゼーション技法 [Gao et al., 2023] と類似するが、材料データベースのドメイン固有カラム意味論（例： `energy_above_hull` は E_{hull} を表す）に特化している。

2.2 材料用語辞書を備えた条件抽出器

条件抽出器は、YAML ベースの日英バイリンガル用語辞書 (`material_terms.yaml`) を用いて、自然言語クエリを構造化された条件辞書に変換する。

2.2.1 辞書構造

辞書は 4 つのカテゴリを含む：

1. プロトタイプ別名：プロトタイプ名とその変種の対応。
例：B2 ↔ [B2, CsCl, CsCl 型, B2 型]
L12 ↔ [L1₂, L12, AuCu₃, Cu₃Au 型, γ']
2. 元素マッピング：元素記号 ↔ 日英名対応。
例：Fe ↔ [Fe, 鉄, iron]

3. 安定性用語：自然言語 → 数値閾値。
「安定」 / “stable” → `energy_above_hull ≤ 0.001 eV/atom`
「準安定」 / “metastable” → `energy_above_hull ≤ 0.05 eV/atom`
4. 物性用語：物性記述からテーブル. カラム参照への対応。
「格子定数」 / “lattice constant” → `structure.lattice_a`
「形成エネルギー」 / “formation energy” → `phase_stability.formation_energy_per_atom`

2.2.2 Unicode 正規化とパターンマッチング

Unicode 下付き文字 (⚡⚡⚡⚡⚡⚡⚡⚡) はマッチング前に ASCII 数字に正規化される。元素記号認識はワード境界正規表現を使用し、独立した「Fe」を「FeCoNi」部分文字列から正しく区別する。

2.2.3 出力形式

抽出器は構造化辞書を返す：

Listing 1: 条件抽出の例。

```

1 # 入力: "Feを含む安定なB2化合物を出して"
2 # 出力:
3 {
4   "prototype": "B2",
5   "contains_elements": ["Fe"],
6   "stability": "stable",
7   "sort_by": None,
8   "sort_order": None
9 }
```

2.3 Schema Graph 走査エンジン

本手法の中核的アルゴリズム貢献である。このエンジンはデータベーススキーマをグラフとして表現し、最適な JOIN 経路を自動計算する。

2.3.1 PostgreSQL メタデータからのグラフ構築

外部キー関係は PostgreSQL の `information_schema` から動的に抽出される：

Listing 2: FK 関係抽出クエリ。

```

1 SELECT tc.table_name AS source_table,
2        kcu.column_name AS source_column,
3        ccu.table_name AS target_table,
4        ccu.column_name AS target_column
5 FROM   information_schema.table_constraints tc
6 JOIN   information_schema.key_column_usage kcu
7       ON tc.constraint_name = kcu.constraint_name
8 JOIN   information_schema.constraint_column_usage ccu
9       ON ccu.constraint_name = tc.constraint_name
10 WHERE tc.constraint_type = 'FOREIGN KEY'
11 AND   tc.table_schema = 'public';
```

結果は NetworkX [Hagberg et al., 2008] の無向グラフ $G = (V, E)$ として格納される。ここで $V = \{\text{テーブル名}\}$ 、 $E = \{\text{FK 関係}\}$ である。FK メタデータは実行時に読み取られるため、コード変更なしにスキーマ変更に対応する。

2.3.2 最小被覆 JOIN 経路：Steiner 木近似

条件抽出器の出力から決定された必要テーブル集合 $R \subseteq V$ に対し、 R のすべてのテーブルを接続する G の最小部分グラフが必要となる。これはグラフにおける Steiner 木問題 [Hwang et al., 1992] であり、一般に NP 困難である。

本スキーマの 7 ノード 6 辺グラフに対し、木構造グラフでは厳密解を与える貪欲ヒューリスティック (アルゴリズム 1) を採用する：

計算量。 $|V| = n$ ノード、 $|R| = k$ 必要テーブルのグラフに対し、アルゴリズムは $O(k^2)$ 回の最短路計算を行い、各計算は $O(n + |E|)$ (BFS) である。本スキーマ ($n = 7, k \leq 5$) ではマイクロ秒で完了する。

Algorithm 1 最小被覆 JOIN 部分グラフ (Steiner 木近似)。

Require: FK 関係グラフ $G = (V, E)$ 、必要テーブル集合 R

Ensure: JOIN 経路リスト P

```
1:  $P \leftarrow \emptyset$ , covered  $\leftarrow \emptyset$ 
2: for all  $(s, t) \in \binom{R}{2}$  do
3:   if  $s \in \text{covered} \wedge t \in \text{covered}$  then
4:     continue
5:   end if
6:    $p \leftarrow \text{ShortestPath}(G, s, t)$ 
7:   if  $p \neq \emptyset$  then
8:      $P \leftarrow P \cup \{p\}$ 
9:     covered  $\leftarrow \text{covered} \cup \text{nodes}(p)$ 
10:  end if
11: end for
12: for all  $r \in R \setminus \text{covered}$  do
13:    $c \leftarrow \text{covered の最近傍ノード}$ 
14:    $P \leftarrow P \cup \{\text{ShortestPath}(G, r, c)\}$ 
15: end for
16: return  $P$ 
```

▷ 両端点は既に接続済み

▷ FK グラフ上の BFS

▷ 孤立必要テーブルの処理

木構造 **FK** グラフでの正確性。 G が木（本スキーマでは `material_entry` がハブ）の場合、Steiner 木は一意であり、貪欲アルゴリズムはこれを厳密に求める。FK サイクルを持つスキーマ（自己参照テーブル等）では近似が不要な辺を含む可能性があるが、SQL 生成においては `DISTINCT` を適用すれば正確性に影響しない。

2.3.3 JOIN 経路 → SQL FROM/JOIN 句生成

P の各経路は SQL JOIN 句に変換される。FK メタデータが結合カラム名を提供する：

Listing 3: 経路からの JOIN 句生成。

```
# 経路: [material_entry, structure]
# FK: structure.entry_id -> material_entry.entry_id
# 生成:
#   FROM material_entry m
#   JOIN structure s ON s.entry_id = m.entry_id
```

テーブル別名はテーブル名の先頭文字から自動割当される（衝突時は解決処理あり）。

2.3.4 多元素 AND クエリ：EXISTS 副問い合わせパターン

組成テーブル正規化に起因する材料固有の重要課題が存在する。「Ni と Al の両方を含む化合物」のクエリには以下が必要：

Listing 4: EXISTS 副問い合わせによる多元素 AND クエリ。

```
1 SELECT DISTINCT m.entry_id, m.formula
2 FROM material_entry m
3 WHERE EXISTS (
4   SELECT 1 FROM composition
5   WHERE entry_id = m.entry_id AND element = 'Ni')
6 AND EXISTS (
7   SELECT 1 FROM composition
8   WHERE entry_id = m.entry_id AND element = 'Al')
9 LIMIT 100;
```

Schema Graph エンジンでは複数元素が指定された場合を検出し、直接 JOIN の代わりに EXISTS 副問い合わせを自動生成する。このパターンなし（Naive アプローチ）では、`WHERE c.element = 'Ni' AND c.element = 'Al'` は単一組成行が両条件を同時に満たすことが不可能なため常に 0 行を返す。これが Schema Graph 手法により防止される最も重要な故障モードである。

2.3.5 比較：Naive vs. Schema Graph（具体例）

表 2 に「Fe を含む B2 化合物を出して」に対する差異を示す：

Table 2: SQL 生成比較：「Fe を含む B2 化合物」に対する Naive vs. Schema Graph。

観点	Naive (Level 0)	Schema Graph (Level 1)
JOIN テーブル	常に全 6 テーブル	composition + structure のみ (2 テーブル)
JOIN 数	5 JOIN	2 JOIN
DISTINCT	なし → 重複行	あり
LIMIT	なし → 無制限	あり (100)
安全性検証	なし	完全 (キーワード + テーブル WL)
多元素 AND	不正 (0 行)	正確 (EXISTS)

2.4 SQL 生成：Rule-based モードと LLM モード

システムは 2 つの SQL 生成モードをサポートし、いずれも Schema Graph 制約内で動作する。

2.4.1 Rule-based モード (Level 1)

決定論的テンプレートベース生成器が、抽出された条件と Schema Graph JOIN 経路から SQL を構築する。外部 API は不要で、レイテンシは 100 ms 未満。制限：数値比較 WHERE 句 (例：band_gap > 1.0) の生成不可、否定 (「Fe を含まない化合物」) の処理不可、辞書未登録用語の理解不可。

2.4.2 Few-Shot RAG を備えた LLM モード (Level 2)

Rule-based モードの辞書カバレッジが不十分な場合 (未登録元素、数値条件、複雑な表現など)、LLM ベース生成にフォールバックする。LLM (本実験では GPT-5) は以下の構造化プロンプトを受け取る：

1. システムメッセージ：タスク説明 + スキーマ YAML (テーブル名、カラム名、型、FK 関係)
2. **Few-Shot** 事例：RAG ストアからの Top- k 類似 NL-SQL ペア (2.5 節)
3. ユーザーメッセージ：自然言語クエリ
4. 制約：「SELECT 文のみ生成」「LIMIT 100 を含める」「スキーマに記載されたテーブルのみ使用」
生成された SQL は実行前に SQL ガード (2.6 節) で検証される。

2.5 Few-Shot RAG ストア (SQL-as-Few-Shot-Examples)

2.5.1 アーキテクチャ

Few-Shot ストアは T2SQL に特化した RAG [Lewis et al., 2020] パターンを実装する：

1. 蓄積：クエリが正常に結果を返した場合、(NL クエリ、SQL、条件、行数、ソース) タプルを JSON ストアに追加する。
2. 検索：新規クエリに対し、全蓄積 NL クエリとの TF-IDF [Salton and Buckley, 1988] コサイン類似度を計算し、上位 k 件 (デフォルト $k = 3$) を返す。
3. 注入：検索された事例を「Question: ... SQL: ...」ブロックとしてフォーマットし、LLM プロンプトの先頭に配置する。

2.5.2 材料クエリのための TF-IDF トークナイゼーション

標準 NLP トークナイザは化学式や元素記号を不正確に分割するため、材料クエリには不適切である。本トークナイザは複合正規表現パターンを使用する：

Listing 5: 材料クエリ用 TF-IDF トークナイザ。

```
# トークナイゼーションパターン：
# 1. 化学記号: [A-Z][a-z]? (Fe, Ni, Al, ...)
# 2. 日本語文字: [\u3040-\u9fff]+
# 3. 英単語: [a-z]+
# 4. 数字: \d+
tokens = re.findall(
    r'[A-Z][a-z]?|[\u3040-\u9fff]+|[a-z]+|\d+',
    query.lower()
)
```

IDF は $IDF(t) = \log(N/n_t)$ で計算される。ここで N は蓄積事例総数、 n_t は語 t を含む事例数である。クエリと蓄積事例の TF-IDF ベクトル間のコサイン類似度によりランキングされる。

2.5.3 研究論文からの種事例抽出

手動キュレーションなしに Few-Shot ストアをブートストラップするため、LaTeX 原稿からの自動種事例抽出を実装した。抽出器は以下のパターンを走査する：

- $B2[-\backslash s]type\backslash s^*(化学式) \rightarrow \{prototype: B2, elements: 抽出\}$
 - $L1[\$_{\{ }2]+[-\backslash s]^*(化学式) \rightarrow \{prototype: L12, elements: 抽出\}$
 - 100 文字以内のコンテキストキーワード：“lattice”, “stable”, “formation energy”
- 抽出された各化合物言及に対し、正規 NL クエリと対応 SQL が生成される。本実験では `hea_lattice_prediction.tex` から 4 件の種事例が自動抽出され、手動キュレーション 7 件と合わせて合計 11 件となった（表 3）。

Table 3: Few-Shot ストア構成（11 事例）。

ソース	件数	比率	クエリ例
手動キュレーション	7	64%	「Fe を含む B2 化合物を出して」
論文抽出	4	36%	「B2 FeAl の物性を出して」

2.6 SQL ガード：多層安全性検証

Rule-based モード・LLM モードを問わず、生成された全 SQL はデータベース実行前に 5 層検証パイプラインを通過する：

1. 複数文検出：SQL 本体内にセミコロンがあれば拒否（`SELECT ...; DROP TABLE ...` 等のピギーバック攻撃を防止）。
 2. 禁止キーワード検出：INSERT, UPDATE, DELETE, DROP, ALTER, TRUNCATE, CREATE, GRANT, REVOKE, COPY をワード境界正規表現で走査。
 3. **SELECT** 専用制約：SELECT または WITH で始まらない文を拒否。
 4. テーブルホワイトリスト検証：FROM/JOIN 句の全テーブル名を抽出し、`allowed_schema.yaml` に存在するか検証。未宣言テーブル（例：`secret_passwords`）参照クエリは拒否。
 5. 自動 **LIMIT** 注入：LIMIT 句がなければ LIMIT 100 を付加し、無制限結果セットによる過剰メモリ・計算消費を防止。
- オプションで SQLGlot [Mao, 2023] が AST（抽象構文木）レベルの構文検証を行う。

2.7 推奨運用構成：カスケードモード

各生成モードの特性に基づき、カスケード戦略を推奨する：

1. 段階 1：Rule-based 生成を試行（Level 1、100 ms 未満）。
 2. 判定：抽出条件が空または辞書カバーレッジが閾値以下の場合、エスカレーション。
 3. 段階 2：Few-Shot RAG 付き LLM 生成にフォールバック（Level 2、約 7 秒）。
 4. 共通：生成モードによらず SQL ガード検証を常に適用。
- このアプローチは、辞書がカバーするクエリ（典型的な材料クエリの推定 70–80%）を外部 API 依存なしに処理し、複雑または新規なクエリにのみ LLM 呼び出しを使用する。

3 データ準備

3.1 OQMD API からのデータ取得

データは OQMD RESTful API (<https://oqmd.org/oqmdapi/formationenergy>) から、2 種類の金属間化合物プロトタイプ構造について取得した：

- **B2 (CsCl 型)**：フィルタ `prototype=CsCl`。重複除去後 636 の固有組成。安定 ($E_{\text{hull}} \leq 0.001$ eV/atom)：185 件。準安定 ($E_{\text{hull}} \leq 0.05$ eV/atom)：198 件。
 - **L1₂ (AuCu₃ 型)**：フィルタ `prototype=AuCu3`。273 の固有組成。安定：88 件。準安定：138 件。
- 合計：909 の固有化合物。各エントリについて 6 フィールドを取得：`name` (化学式)、`entry_id`、`prototype`、`delta_e` (原子当たり形成エネルギー)、`stability` (hull 上エネルギー)、`band_gap`。

API はページネーションされた JSON レスポンス (`limit=200, offset=0,200,...`) を返す。重複エントリ（同一化学式、異なる `entry_id`）は最低エネルギーのものを保持して重複除去した。

3.2 RDB へのデータ投入

各 OQMD エントリを 7 テーブルスキーマに分解する。例えば、 Fe_3Al (L1_2 プロトタイプ) は以下を生成する: `material_entry` に 1 行、`composition` に 2 行 (Fe: 0.75、Al: 0.25)、`structure` に 1 行 (prototype=L12、lattice_a、space_group)、`phase_stability` に 1 行 (formation_energy、energy_above_hull、band_gap)、`calculation` に 1 行 (method=GGA)。

3.3 スキーマ拡張

元スキーマに含まれない OQMD フィールドを収容するため以下のカラムを追加した: `phase_stability` に `band_gap`(REAL)、`structure` に `space_group`(TEXT)。材料用語辞書に対応用語 (band_gap、volume_per_atom、space_group) を追加した。

4 結果

4.1 テストケース設計

正確性、安全性、頑健性を評価するため、6 カテゴリ 39 件のテストケースを設計した (表 4)。

Table 4: テストケースカテゴリ (全 39 件)。

カテゴリ	ID	n	検証焦点
正常系	A01–A15	15	元素、プロトタイプ、安定性、ソート、複合条件
該当なし	B01–B05	5	希ガス/貴ガス元素、非存在組合せ → 0 行期待
いい加減な入力	C01–C10	10	空入力、無関係テキスト、単語のみ、数値条件
矛盾条件	D01–D02	2	「安定かつ準安定」「Fe なし Fe 化合物」
SQL インジェクション	E01–E05	5	DROP, DELETE, INSERT、ビジーバック攻撃
安全性検査	F01–F02	2	直接 SQL 入力 (SELECT, UPDATE)

4.2 3 水準比較

図 3 および表 5 に 3 水準の比較結果を示す。

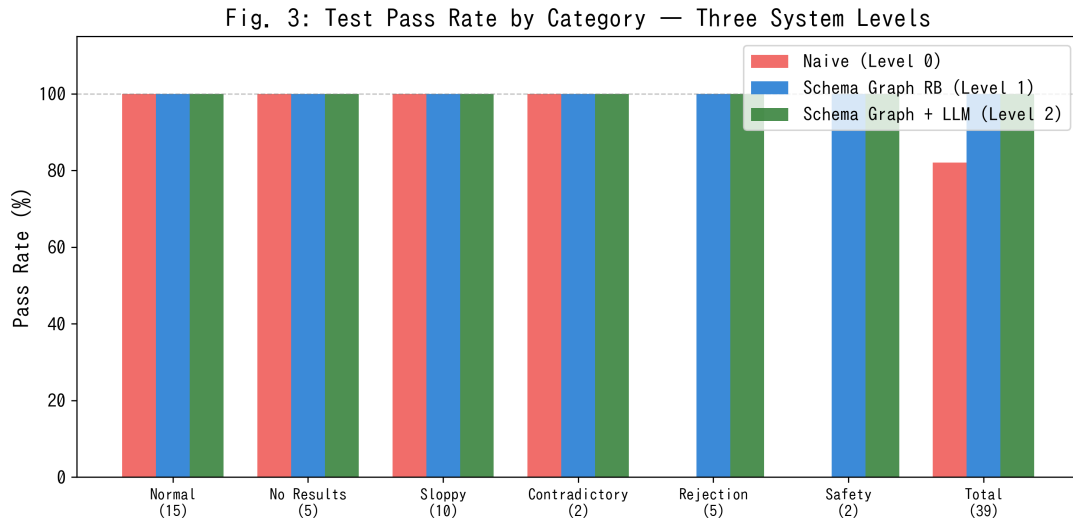


Figure 3: 3 水準のカテゴリ別テストパス率。Naive (Level 0) は SQL インジェクションと安全性カテゴリで不合格 (SQL ガードなし)。Schema Graph 搭載の両水準 (1, 2) は全カテゴリ 100%を達成。

Table 5: 3 水準の定量比較。

	Naive (L0)	SG + RB (L1)	SG + GPT-5 (L2)
パス率	32/39 (82%)	39/39 (100%)	39/39 (100%)
平均レイテンシ	<100 ms	<100 ms	~7,300 ms
トークン消費	0	0	49,103 (39 クエリ)
不要 JOIN 除去	不可	可能	可能
多元素 AND	不正確	正確	正確
SQL インジェクション遮断	不可	可能	可能
未登録語彙処理	無視	無視	理解可能
数値 WHERE 生成	不可	不可	可能
外部 API 依存	なし	なし	OpenAI

4.3 Rule-based vs. GPT-5 詳細比較

4.3.1 結果セット一致度 (Jaccard 指数)

比較可能な 32 件のテスト (インジェクション/安全性カテゴリ除く) について、Rule-based と GPT-5 の結果セット間の Jaccard 類似度を計算した (図 4)。

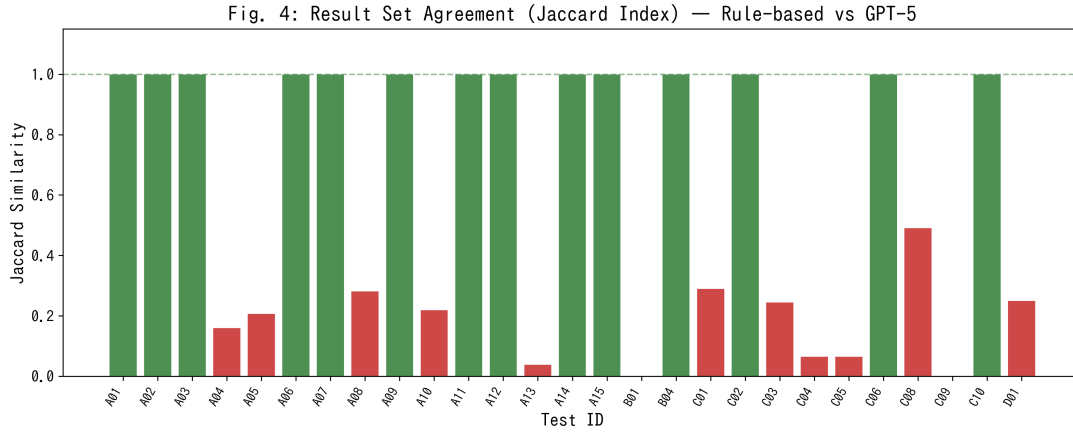


Figure 4: Rule-based と GPT-5 結果セット間の Jaccard 指数。緑: ≥ 0.8 (高一致)、橙: $0.5-0.8$ (中程度)、赤: < 0.5 (低)。低一致ケースは主に GPT-5 の優れた意味理解に起因 (例: B01: Xe 化合物、C09: NiAl 特定)。

4.3.2 注目すべき乖離

GPT-5 が Rule-based を上回る 3 ケースを示す:

B01: 「Xe を含む B2 化合物を出して」 Rule-based: Xe は辞書未登録 → 条件無視 → 全 B2 化合物を返却 (100 行、偽陽性)。GPT-5: 正しく `WHERE element = 'Xe'` を生成 → 2 行 (CsXe, MgXe)。

C09: 「NiAl L12」 Rule-based: Ni, Al, L12 を独立に抽出 → Ni または Al を含む全 L1₂ (100 行)。GPT-5: 「NiAl」を特定化合物と理解 → AlNi₃ のみ (1 行)。

D02: 「Fe なし Fe B2 化合物」 Rule-based: 否定を解析不可 → Fe 含有 B2 化合物を返却。GPT-5: 矛盾を認識 → 適切なクエリを生成。

4.3.3 レイテンシとコスト

図 6 にクエリごとのレイテンシを示す。Rule-based モードは一貫して 100 ms 未満。GPT-5 モードは平均 7,305 ms (最大 23,348 ms)、約 70 倍の増加。39 クエリの総トークン消費は 49,103 トークン (約 1,259 トークン/クエリ)。

4.4 OQMD API 正解データによる検証 (結合テスト)

8 件のテストケースについて、同等のフィルタ条件で OQMD REST API に直接問い合わせ、T2SQL 出力と比較した (表 6、図 7)。Precision = $|T2SQL \cap OQMD| / |T2SQL|$ 、Recall = $|T2SQL \cap OQMD| / |OQMD|$ 。

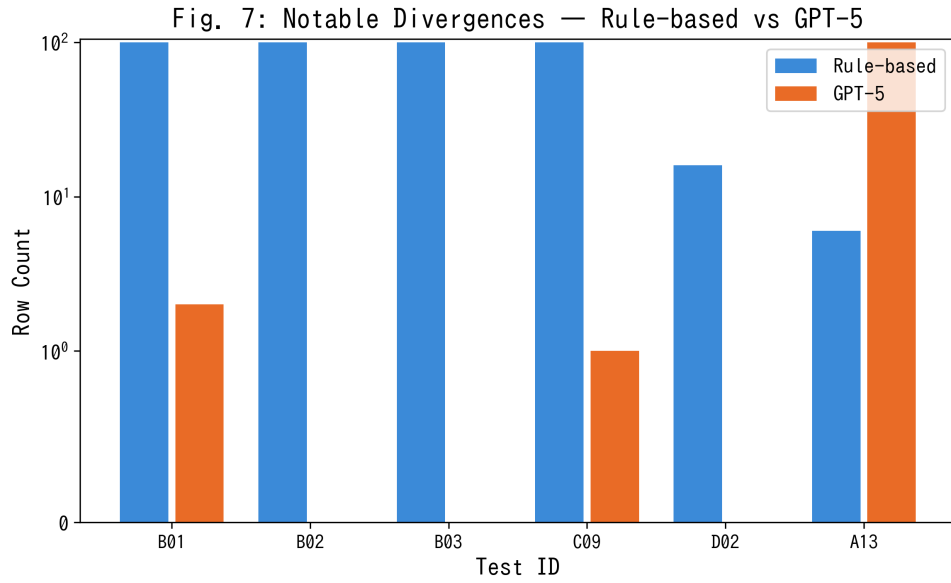


Figure 5: 注目乖離ケースの行数比較。B01–B03 : Rule-based は未登録元素により過剰結果を返却、GPT-5 は正確な（少ない）結果セットを返却。

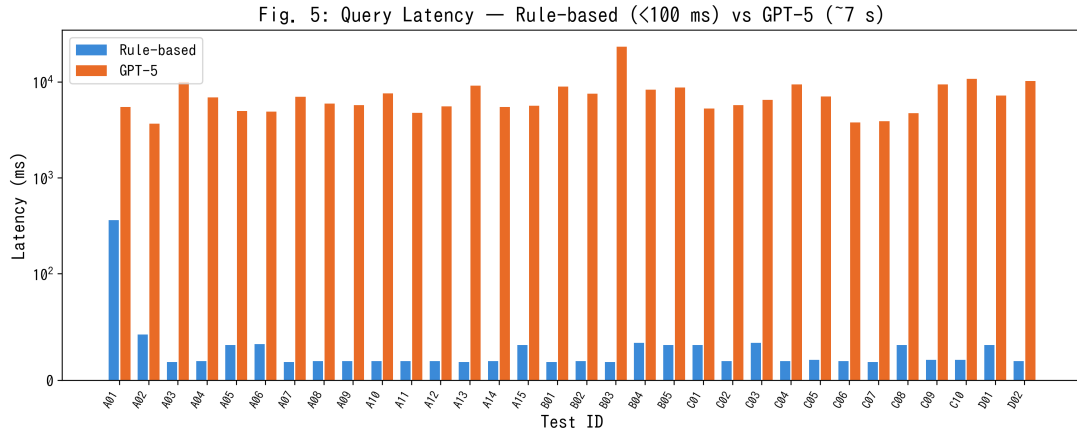


Figure 6: クエリごとのレイテンシ比較（対数スケール）。Rule-based : 一貫して 100 ms 未満。GPT-5 : 平均約 7.3 秒。

Table 6: OQMD API 正解データとの比較 (Precision / Recall)。

ID	クエリ条件	OQMD	T2SQL	Prec.	Rec.	備考
A01	Fe + B2	7	7	1.000	1.000	完全一致
A02	安定 L1 ₂ (ソート済)	88	88	1.000	1.000	完全一致
A04	全 B2	483	95	1.000	0.197	LIMIT 制約
A05	準安定 B2	260	90	1.000	0.346	LIMIT 制約
A06	Co + 安定 L1 ₂	1	1	1.000	1.000	完全一致
A10	全 L1 ₂	273	100	1.000	0.366	LIMIT 制約
A15	Ni + 安定 L1 ₂	6	6	1.000	1.000	完全一致
C08	安定 B2、band_gap > 0	0	78	0.000	—	API フィルタ非対応

Fig. 6: OQMD-API Ground Truth Comparison — Precision & Recall

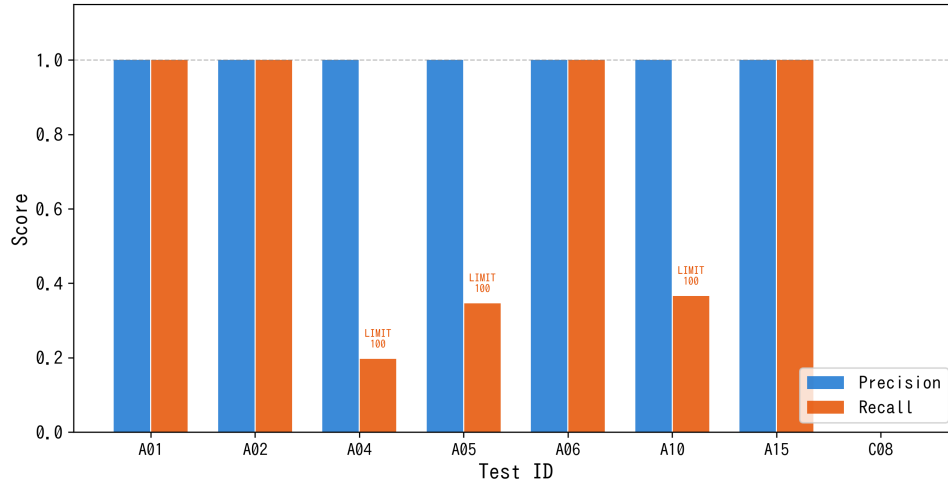


Figure 7: OQMD API 正解データに対する Precision (青) と Recall (橙)。C08 を除く全テストで Precision = 1.0 を達成。A04/A05/A10 の低 Recall はデータ不正確性ではなく LIMIT 100 制約による。

結合テストとしての意義。この比較は 5 段階のデータ変換パイプライン全体を検証する：(1) API JSON → CSV 変換、(2) CSV → 7 テーブル正規化、(3) NL → SQL 変換 (条件抽出 + Schema Graph)、(4) JOIN 経路の正確性 (FK グラフ走査)、(5) SQL 実行 (DISTINCT/LIMIT)。Precision = 1.0 は、これら 5 段階にわたってデータ損失・破損・不正 JOIN が発生していないことを確認する。いずれかの段階にバグがあれば—例えば、組成テーブルでの元素分割誤り、FK 整合性違反、JOIN 経路欠落—Precision は 1.0 を下回る。

注意：同一データソース (OQMD) がデータベース内容と正解データの両方に使用されているため、この検証はシステムの未知データベースへの汎化能力の独立評価とはならない。異なるデータベース (Materials Project 等) との交差検証は今後の課題である。

4.5 SQL インジェクションと安全性結果

7 件の敵対的入力 (E01–E05, F01–F02) はすべて SQL ガードにより遮断された (表 7)。

Table 7: SQL インジェクション・安全性テスト結果 (全件遮断)。

ID	入力	遮断理由
E01	DROP TABLE material_entry;	禁止キーワード：DROP
E02	SELECT *; DELETE FROM composition;	複数文 + DELETE
E03	B2 化合物; DROP TABLE structure;	複数文 + DROP
E04	SELECT * FROM secret_passwords	非許可テーブル
E05	INSERT INTO material_entry ...	禁止キーワード：INSERT
F01	SELECT entry_id FROM material_entry	LIMIT なし → 自動付加
F02	UPDATE material_entry SET ...	禁止キーワード：UPDATE

5 考察

5.1 多次元評価

図 8 に 3 水準の 6 次元レーダーチャート比較を示す：精度、レイテンシ (逆数)、コスト (逆数)、安全性、語彙カバレッジ、数値フィルタ能力。

Fig. 8: Multi-dimensional Comparison of Three System Levels

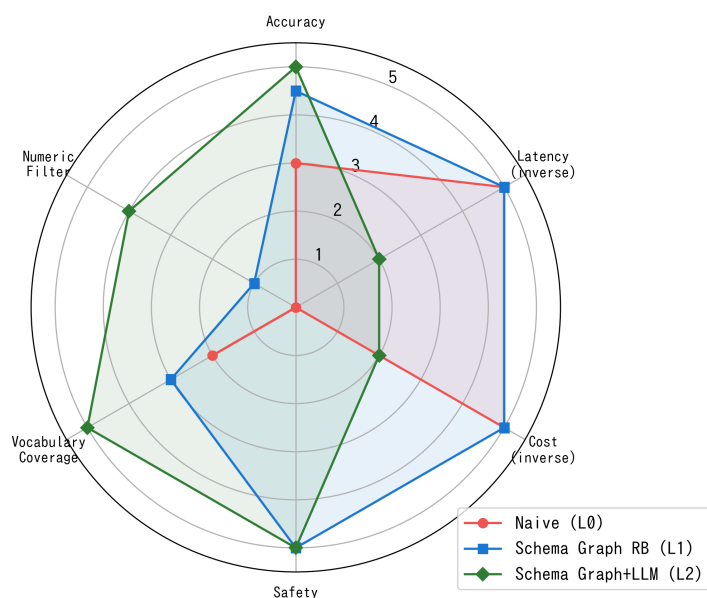


Figure 8: 3 水準の多次元比較（各軸 5 点満点）。Schema Graph + LLM（Level 2）は最も包括的だが、レイテンシとコストを犠牲にする。

Table 8: 各水準の利点と欠点。

水準	利点	欠点
Naive (L0)	最小実装（約 50 行）。依存なし。最速（100 ms 未満）	不要 JOIN。多要素 AND 失敗。安全性なし。LIMIT/DISTINCT なし
SG + RB (L1)	外部 API 不要。100 ms 未満。完全な安全性。100% 決定論的。再現可能	未登録語彙で無通知失敗。数値 WHERE 不可。手動語彙拡張が必要
SG + LLM (L2)	未登録語彙対応。数値/否定可能。文脈理解	外部 API 依存（コスト、レイテンシ、プライバシー）。非決定論的。幻覚リスク

5.2 各水準の利点と欠点

5.3 限界と制約

5.3.1 テストケースの自己参照性（深刻度：高）

39 件のテストケースは抽出パターンを知るシステム開発者が設計した。したがって、制御された条件下で 100% のパス率は予想される。真の精度評価には材料研究者による自由文クエリのブラインドテストが必要である。

5.3.2 Few-Shot RAG 効果の未検証（深刻度：高）

Few-Shot ストアの主要機能—類似事例注入による LLM SQL 生成精度向上—はアブレーション研究で分離されていない。今後の課題として、ホールドアウトテストセットにおける Few-Shot 事例の有無による LLM 性能比較が必要である。

5.3.3 無通知失敗（深刻度：中）

Rule-based モードで未登録語彙が出現した場合、条件が無通知で破棄され、ユーザーへの通知なしに過度に広範な結果を生成する。クエリ語彙が部分的に未認識であることをユーザーに警告する「条件カバレッジスコア」の実装がこのリスクを軽減する。

5.3.4 小規模スキーマ（深刻度：中）

本研究の 7 テーブル・909 レコードスキーマは本番データベース（OQMD：100 万件超、テーブル数十の可能性）より遥かに小さい。Schema Graph 走査の Steiner 木近似は木構造グラフで厳密だが、FK サイクルを持つ大規模スキーマでは性能劣化の可能性がある。20 テーブル以上・10 万レコード以上のスケーラビリティテストが必要である。

5.3.5 Rule-based モードの数値条件制限（深刻度：中）

Rule-based モードは数値比較句（例：`band_gap > 1.0`）を生成できない。これは材料スクリーニングで頻繁に必要とされる。不等式演算子と値の正規表現ベース抽出（数値条件パーサー）により、LLM 呼び出しなしでこの問題を解決できる。

5.4 関連システムとの比較

Table 9: 関連する材料 NLP / T2SQL システムとの比較。

システム	スキーマ認識	正規化 RDB	材料特化	Few-Shot	主な差異
Spider [Yu et al., 2018]	あり	あり	なし	なし	汎用ベンチマーク
BIRD [Li et al., 2024]	あり	あり	なし	なし	大規模実データ
DAIL-SQL [Gao et al., 2023]	あり	あり	なし	あり	スケルトン選択
MKNA [Zhuang and Farimani, 2026]	なし	なし (API)	あり	なし	エージェント、SQL 生成なし
OptiMat [Hu and Turlo, 2026]	なし	なし	あり	なし	オンデマンド DFT
MP MCP	部分的	なし (API)	あり	なし	MCP プロトコル
本手法	あり	あり	あり	あり	FK グラフ + 材料辞書

5.5 AI for Science (AI4S) における意義

本手法は AI4S パラダイムに直接的な含意を持つ。表 10 に AI4S ワークフローの各段階と T2SQL 手法の役割を示す。

重要な点として、Schema Graph アーキテクチャはデータベース非依存である：FK 関係は実行時に PostgreSQL メタデータからイントロスペクション（2.3 節）されるため、同じエンジンを任意の正規化材

Table 10: AI4S 材料探索ワークフローにおける T2SQL の役割。

AI4S 段階	従来アプローチ	T2SQL 手法適用時
仮説生成	研究者が手動でクエリを作成	LLM エージェントが仮説から NL クエリを生成
データ検索	SQL または API 呼び出しを手動記述	NL → SQL 自動変換 (Schema Graph 経由)
スクリーニング	反復的 SQL 修正	Few-Shot RAG が成功クエリパターンを再利用
結果解釈	クエリ結果の手動検査	メタデータ付き構造化出力 (プロトタイプ、安定性、物性)
フィードバック	知識は個々の研究者に留まる	成功クエリが Few-Shot ストアに蓄積され再利用

料データベース (Materials Project、AFLOW、NOMAD、機関内データベース) にコード変更なしで展開できる—用語辞書のドメイン適応のみが必要である。このポータビリティは AI4S にとって不可欠であり、異種データソースを統一インターフェースでクエリする必要がある。

さらに、カスケード Rule-based → LLM アーキテクチャ (2.7 節) は、AI4S が要求する信頼性とコスト効率の要件に適合する：定型クエリは外部 API 呼び出しなしに決定論的に処理され、複雑なクエリのみが LLM 能力を活用する。このハイブリッドアプローチにより、AI4S パイプラインは禁止的な API コストやレイテンシなしに大規模運用が可能になる。

5.6 今後の展望

1. マルチデータベース拡張：Schema Graph を Materials Project、AFLOW スキーマに適応。動的 FK イントロスペクションはアーキテクチャ上これを既にサポートしている。
2. **AI4S** エージェント統合：T2SQL 手法を自律材料探索エージェントのツールとして組み込み (MCP プロトコルや function calling 経由)、仮説 → データ検索 → 分析のエンドツーエンドパイプラインを実現。
3. ブラインドユーザースタディ：10 名以上の材料研究者から自由文クエリを収集。
4. **Few-Shot** アプレーション研究：LLM SQL 品質への Few-Shot RAG の影響を定量化。
5. 対話的クエリ精緻化：無通知フォールバックの代わりに曖昧クエリに対する確認質問を返却。
6. 数値条件パーサー：不等式条件の Rule-based 抽出。
7. 大規模スキーマ検証：20 テーブル以上、10 万レコード以上。
8. クロスリンガル埋め込み検索：TF-IDF を transformer 埋め込みに置換し Few-Shot 類似度を改善—国際 AI4S 協力において多言語クエリが到着する前提条件。

6 結論

本研究では、909 件の OQMD 金属間化合物 (B2 および L1₂ 型) で検証した、材料データベースのための Schema Graph 支援 Text-to-SQL 手法を提示した。本手法は材料インフォマティクスの特定のギャップ—材料ドメイン語彙を備えた正規化リレーショナルデータベース上のスキーマ認識型 **SQL** 生成—に取り組む。

主要な知見：

1. **Schema Graph** 走査は不要 JOIN を排除し、多元素 AND 検索のための EXISTS 副問い合わせを正しく生成する—正規化された組成テーブルにおける最も重要な故障モードの防止。
2. **Rule-based** モードは外部 API 依存ゼロで 100 ms 未満のレイテンシにて 39/39 テストパス率を達成し、辞書カバー対象クエリに適する。
3. **LLM** モード (**GPT-5**) は未登録語彙、数値条件、否定への対応を約 70 倍のレイテンシコストで拡張する。
4. **OQMD API** 正解データ比較はデータパイプライン全体 (API → RDB → NL → SQL → 結果) に対して Precision = 1.0 を確認し、結合テストの正確性を検証する。
5. **SQL** ガードはテスト対象の全インジェクション攻撃を遮断し、結果セット上限を強制する。

カスケード Rule-based → LLM アーキテクチャは速度・コスト・カバレッジのバランスを取り、材料研究者が SQL 専門知識なしに正規化データベースを自然言語でクエリすることを可能にする。

AI for Science (AI4S) のより広い文脈において、本研究は重要なインフラストラクチャのギャップに取り組む：自然言語の意図を実行可能なデータベースクエリへ確実に変換する能力は、自律 AI 駆動型材料探索ワークフローの前提条件である。AI が研究者とデータの間に仲介する度合いが増す中、Schema Graph T2SQL 手法は、実行時 FK イントロスペクションにより異種材料データベース (OQMD、Materials Project、AFLOW、機関内 DB) に展開可能なポータブルかつデータベース非依存の基盤を提供する。Few-Shot RAG フィードバックループはさらに再学習なしの継続的改善を可能にし、AI4S システムの自己改善的性質に適合する。

現在の検証は規模とテストケース独立性において限定的であるが、本手法は材料データベース特化型 T2SQL の再現可能なフレームワークを確立し、より大規模なスキーマ、追加データソース、AI4S エージェントアーキテクチャへの統合へと拡張可能である。

謝辞

DFT 計算データは Open Quantum Materials Database (OQMD) から取得した。OQMD の開発・維持管理にあたる Northwestern 大学 Wolverton グループに感謝する。本研究は NIMS の支援を受けた。

References

- Hanchen Wang, Tianfan Fu, Yuanqi Du, Wenhao Gao, Kexin Huang, Ziming Liu, Payal Chandak, Shengchao Liu, Peter Van Katwyk, Andreea Deac, Anima Anandkumar, Katja Bergen, Carla P. Gomes, Shirley Ho, Pushmeet Kohli, Joan Lasenby, Jure Leskovec, Tie-Yan Liu, Arjun Manber, Debapratim Misra, Eliot Moret, Joelle Pineau, Ben Poole, Marina Sacks, Saayan SenGupta, Jian Sun, Jie Tang, Adam Trischler, Max Welling, Yaochen Xie, and Marinka Zitnik. Scientific discovery in the age of artificial intelligence. *Nature*, 620:47–60, 2023. doi: 10.1038/s41586-023-06221-2.
- James E. Saal, Scott Kirklin, Muratahan Aykol, Bryce Meredig, and Christopher Wolverton. Materials design and discovery with high-throughput density functional theory: The Open Quantum Materials Database (OQMD). *JOM*, 65:1501–1509, 2013. doi: 10.1007/s11837-013-0755-4.
- Scott Kirklin, James E. Saal, Bryce Meredig, Alex Thompson, Jeff W. Doak, Muratahan Aykol, Stephan Rühl, and Christopher Wolverton. The Open Quantum Materials Database (OQMD): Assessing the accuracy of DFT formation energies. *npj Comput. Mater.*, 1:15010, 2015. doi: 10.1038/npjcompumats.2015.10.
- Anubhav Jain, Shyue Ping Ong, Geoffroy Hautier, Wei Chen, William Davidson Richards, Stephen Dacek, Shreyas Cholia, Dan Gunter, David Skinner, Gerbrand Ceder, and Kristin A. Persson. Commentary: The Materials Project: A materials genome approach to accelerating materials innovation. *APL Mater.*, 1:011002, 2013. doi: 10.1063/1.4812323.
- Stefano Curtarolo, Wahyu Setyawan, Gus L. W. Hart, Michal Jahnatek, Roman V. Chepulskii, Richard H. Taylor, Shidong Wang, Junkai Xue, Kesong Yang, Ohad Levy, Michael J. Mehl, Harold T. Stokes, Denis O. Demchenko, and Dane Morgan. AFLOW: An automatic framework for high-throughput materials discovery. *Comput. Mater. Sci.*, 58:218–226, 2012. doi: 10.1016/j.commatsci.2012.02.005.
- Claudia Draxl and Matthias Scheffler. NOMAD: The FAIR concept for big data-driven materials science. *MRS Bull.*, 43:676–682, 2018. doi: 10.1557/mrs.2018.208.
- Oleg N. Senkov, Daniel B. Miracle, Kevin J. Chaput, and Jean-Philippe Couzinie. Development and exploration of refractory high entropy alloys — A review. *J. Mater. Res.*, 33:3092–3128, 2018. doi: 10.1557/jmr.2018.153.
- Easo P. George, Dierk Raabe, and Robert O. Ritchie. High-entropy alloys. *Nat. Rev. Mater.*, 4:515–534, 2019. doi: 10.1038/s41578-019-0121-4.
- Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In *Proceedings of EMNLP*, pages 3911–3921, 2018.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? A BIG Bench for Large-Scale Database Grounded Text-to-SQL (BIRD). *Advances in Neural Information Processing Systems*, 36, 2024.
- Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-SQL empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.

- Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In *Advances in Neural Information Processing Systems*, volume 36, 2024.
- Zijin Hong, Zheng Yuan, Qinggang Zhang, Hao Chen, Junnan Dong, Feiran Huang, and Xiao Huang. Next-generation database interfaces: A survey of LLM-based text-to-SQL. *arXiv preprint arXiv:2406.08426*, 2024.
- Karime Maamari, Fadhil Abubaker, Daniel Jaroslawicz, and Amine Mhedhbi. The death of schema linking? text-to-SQL in the age of well-reasoned language models. *arXiv preprint arXiv:2408.07702*, 2024.
- Yu Song, Santiago Miret, and Bang Liu. MatSci-NLP: Evaluating scientific language models on materials science language tasks using text-to-schema modeling. In *Proceedings of the 61st Annual Meeting of the ACL*, pages 3621–3639, 2023.
- Vaibhav Mishra, Somaditya Singh, Mohd Zaki, et al. LLAMAT: Large language models for materials science information extraction. *arXiv preprint arXiv:2411.00177*, 2024.
- Genmao Zhuang and Amir Barati Farimani. From natural language to materials discovery: The Materials Knowledge Navigation Agent. *arXiv preprint arXiv:2602.11123*, 2026.
- Yang Hu and Vladyslav Turlo. OptiMat Alloys: A FAIR end-to-end agent with living database for computational multi-principal alloy exploration. *arXiv preprint arXiv:2604.21850*, 2026.
- Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX, 2008. Proceedings of the 7th Python in Science Conference (SciPy).
- Frank K. Hwang, Dana S. Richards, and Pawel Winter. *The Steiner Tree Problem*. North-Holland, 1992.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.
- Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Inf. Process. Manage.*, 24:513–523, 1988. doi: 10.1016/0306-4573(88)90021-0.
- Toby Mao. SQLGlot: A python SQL parser, transpiler, optimizer, and engine. <https://github.com/tobymao/sqlglot>, 2023.

A 全テストケース一覧

ID	カテゴリ	自然言語クエリ	結果
A01	正常系	Fe を含む B2 化合物を出して	PASS
A02	正常系	安定な L1 ₂ 化合物を形成エネルギー順に出して	PASS
A03	正常系	Ni と Al の両方を含む化合物を出して	PASS
A04	正常系	B2 化合物の全リストを出して	PASS
A05	正常系	準安定な B2 化合物を出して	PASS
A06	正常系	Co を含む安定な L1 ₂ 化合物を出して	PASS
A07	正常系	Fe と Al を含む B2 と L1 ₂ 化合物を出して	PASS
A08	正常系	γ' 化合物を出して	PASS
A09	正常系	ニッケルを含む L1 ₂ 型化合物を出して	PASS
A10	正常系	L1 ₂ 化合物の全データを出して	PASS
A11	正常系	Ti を含む B2 化合物を格子定数順 (降順) に出して	PASS
A12	正常系	CsCl 型化合物を出して	PASS
A13	正常系	Cu ₃ Au 型化合物を出して	PASS
A14	正常系	鉄を含む B2 化合物を出して	PASS
A15	正常系	安定な L1 ₂ 化合物で Ni を含むものを出して	PASS
B01	該当なし	Xe を含む B2 化合物を出して	PASS
B02	該当なし	U と Pu を含む L1 ₂ 化合物を出して	PASS
B03	該当なし	Rn と Og を含む B2 化合物を出して	PASS
B04	該当なし	Sc と Ir を含む安定な B2 化合物を出して	PASS
B05	該当なし	Pt と Ga を含む安定な B2 化合物を出して	PASS
C01	いい加減	安定な化合物を出して	PASS
C02	いい加減	B2	PASS
C03	いい加減	安定なものを出して	PASS
C04	いい加減	(空入力)	PASS
C05	いい加減	今日の天気を教えて	PASS
C06	いい加減	Fe	PASS
C07	いい加減	格子定数	PASS
C08	いい加減	band gap が大きい安定な B2 化合物	PASS
C09	いい加減	NiAl L12	PASS
C10	いい加減	Fe の B2 か L12	PASS
D01	矛盾	安定かつ準安定な B2 化合物	PASS
D02	矛盾	Fe なしの Fe B2 化合物	PASS
E01	拒否	DROP TABLE material_entry;	PASS
E02	拒否	SELECT *; DELETE FROM composition;	PASS
E03	拒否	B2 化合物; DROP TABLE structure;	PASS
E04	拒否	SELECT * FROM secret_passwords	PASS
E05	拒否	INSERT INTO material_entry VALUES (...)	PASS
F01	安全性	SELECT entry_id FROM material_entry	PASS
F02	安全性	UPDATE material_entry SET formula='X'	PASS

B 生成SQL例

B.1 A01: 「Feを含むB2化合物を出して」

Naive (Level 0): 「

```
1 SELECT m.entry_id, m.formula, s.prototype
2 FROM material_entry m
3     JOIN composition c ON c.entry_id = m.entry_id
4     JOIN structure s ON s.entry_id = m.entry_id
5     JOIN phase_stability ps ON ps.entry_id = m.entry_id
6     JOIN calculation calc ON calc.entry_id = m.entry_id
7     JOIN calculated_property cp
8         ON cp.calculation_id = calc.calculation_id
9 WHERE s.prototype = 'B2' AND c.element = 'Fe';
10 -- 問題: 不要JOIN 3件、LIMIT/DISTINCTなし
```

Schema Graph + Rule-based (Level 1): 「

```
1 SELECT DISTINCT m.entry_id, m.formula,
2     s.prototype, s.lattice_a, s.space_group
3 FROM material_entry m
4     JOIN composition c ON c.entry_id = m.entry_id
5     JOIN structure s ON s.entry_id = m.entry_id
6 WHERE (s.prototype = 'B2' OR s.strukturbericht = 'B2')
7     AND c.element = 'Fe'
8 LIMIT 100;
9 -- 最適: 必要な2 JOINのみ、DISTINCT、LIMIT付き
```

Schema Graph + GPT-5 (Level 2): 「

```
1 SELECT DISTINCT m.entry_id, m.formula,
2     s.prototype, s.lattice_a, s.space_group
3 FROM material_entry m
4 JOIN structure s ON s.entry_id = m.entry_id
5 JOIN composition c ON c.entry_id = m.entry_id
6 WHERE (s.prototype = 'B2' OR s.strukturbericht = 'B2')
7     AND c.element = 'Fe'
8 LIMIT 100;
9 -- LLMがSchema Graph制約内で同等SQLを生成
```

B.2 A03: 「NiとAlの両方を含む化合物を出して」

Naive (Level 0): 「

```
1 SELECT m.entry_id, m.formula
2 FROM material_entry m
3     JOIN composition c ON c.entry_id = m.entry_id
4     ...(全テーブルJOIN)...
5 WHERE c.element = 'Ni' AND c.element = 'Al';
6 -- 致命的: 単一行が両条件を同時に満たすことは不可能
7 -- -> 常に0行を返す
```

Schema Graph + Rule-based (Level 1): 「

```
1 SELECT DISTINCT m.entry_id, m.formula
2 FROM material_entry m
3 WHERE EXISTS (
4     SELECT 1 FROM composition
5     WHERE entry_id = m.entry_id AND element = 'Ni')
6 AND EXISTS (
7     SELECT 1 FROM composition
8     WHERE entry_id = m.entry_id AND element = 'Al')
9 LIMIT 100;
10 -- EXISTS副問い合わせによる正確な多元素AND
```